1. Introduction to PPO and SAC

Proximal Policy Optimization (PPO) [1] and Soft Actor-Critic (SAC) [2] are two widely used deep reinforcement learning algorithms which are introduced in the late 2010s. These algorithms are use for continous control but they differ in how they use the *data*. PPO is an on-policy method, which means that each policy iteration update is computed using tragectories collected by the current policy, and when the policy is updated all previously gathered data is discarded. SAC, on the other hand, is an off-policy method, it stores the previous transitions in a replay buffer, meaning that it can continue to learn from older transitions stored in the buffer, which usually makes it much more sample efficient.

PPO is a gradient method that seek to take the largest possible improvement step with destablizing the policy.

Proximal Policy Optimization [1] is a gradient method that optimizes a clipped surrogate objective function. It was a direct improvement over the previously proposed Trust Region Policy Optimization's (TRPO) pitfalls due to the involvement of KL Divergence as a constraint over optimizing the surrogate objective function. PPO proposes the use of a clip function to prevent large policy updates instead of KL divergence penalty or constraint. "PPO outperforms other online policy gradient methods, and overall strikes a favorable balance between sample complexity, simplicity, and wall-time"[1].

Soft Actor Critic (SAC) is an off-policy actor-critic RL algorithm based on the maximum entropy reinforcement leanring framework. The soft actor-critic algorithm incorporates three key ingredients: an actor-critic architecture with separate policy and value function networks, an off-policy formulation that enables reuse of previously collected data for efficiency, and entropy maximization to enable stability and exploration [2]. It consists of an actor trying arrive at an optimal policy and a critic that evaluates the generated policy. Both actor critic tend to get better with training, critic building more accurate estimations of state value and Q functions and the actor generating policies with higher returns. The algorithm maintains four different set of weights as described ahead: ψ for a soft value function approximator, $\boldsymbol{V_\psi}$ that essentially estimates state value functions of various states, θ for Q value network or critic, $\boldsymbol{Q_\theta}$, that essentially estimates Q-function values for various state action pairs, φ for the policy network or actor,$\boldsymbol{\pi_\varphi}$, that generates policies and finally $\overline{\psi}$ which represents a target value function $\boldsymbol{V_{\overline{\psi}}}$ and is updated as a moving average of the weights of the soft value function approximator V_ψ. The following loss functions were proposed in the original paper and the weights are updated by minimising over the mentioned loss functions.

$$J_V(\psi) = E_{s_t \sim D}\left[\frac{1}{2}\Big(V_\psi(s_t) - E_{a_t \sim \pi_\varphi}[Q_\theta(s_t, a_t) - \log \pi(a_t|s_t)]\Big)^2\right] \tag{1}$$

$$\begin{aligned} J_Q(\theta) &= E_{(s_t,a_t)\sim D}\left[\frac{1}{2}\Big(Q_\theta(s_t, a_t) - \hat{Q}(s_t, a_t)\Big)^2\right] \text{ where,} \\ \hat{Q}(s_t, a_t) &= r(s_t, a_t) + \gamma E_{s_{t+1}\sim p}\Big[V_{\overline{\psi}}(s_{t+1})\Big] \end{aligned} \tag{2}$$

$$J_\pi(\varphi) = E_{s_t \sim D, \varepsilon_t \sim N}\Big[\log \pi_\varphi\Big(f_\varphi(\varepsilon_t; s_t)|s_t\Big) - Q_\theta(s_t, f(\varepsilon_t; s_t))\Big] \tag{3}$$

$$\overline{\psi} \leftarrow \tau\psi + (1-\tau)\psi \tag{4}$$

However in the code implementation we don't maintain a separate soft value network and directly train ψ on critic losses replacing θ in Equation 2. Essentially Equation 2 can be rewritten as the following:-

$$\begin{aligned} J_Q(\psi) &= E_{(s_t,a_t)\sim D}\left[\frac{1}{2}\Big(Q_\psi(s_t, a_t) - \hat{Q}(s_t, a_t)\Big)^2\right] \text{ where,} \\ \hat{Q}(s_t, a_t) &= r(s_t, a_t) + \gamma E_{s_{t+1}\sim p}\Big[V_{\overline{\psi}}(s_{t+1})\Big] \end{aligned} \tag{5}$$

The target netwrok weights $\overline{\psi}$ are now calculated directly as a moving average of the critic weights.

A neat little trick that was proposed in the original paper and is also a part of the code implementation that is worth noting is that, we actually train two sets of Q-function network weights $\{\psi_1, \psi_2\}$ that are trained independently to mitigate positive bias. The minimum of the two Q-functions is then utilised in Equation 1 and Equation 3.

SAC was an improvement over previously proposed RL methods in terms of sample efficiency as it is an off-policy method and stability to convergence and sensitivity to hyperparameters which was notoriously tough to achieve with off-policy model free methods.

2. Relevance to Robotics

Robotics pose specific set of challenges originating from real world setting that make using classical RL techniques infeasible. Methods like PPO and SAC mitigate these challenges quite well making them a good choice for robotics tasks. For instance, PPO and SAC can support large sizes of state space and action space and can generalize well to scenarios that these models have not encountered enough while training. These methods also work quite well with continuous action and state spaces making them an ideal choice for robotics. PPO and SAC both show fair amount of resilience towards noisy inputs through clipping policy update preventing large policy shifts in PPO and entropy regularized objective reducing over reliance on any single observation in SAC.

3. Environment Description

We chose to train in the Ant-v4 and HalfCheetah-v4 environments in MuJoCo gymnasium [3]. These were chosen because the represent different complexities of the same locomotion goal. This also makes them directly comparable for benchmarking PPO and SAC.

The **HalfCheetah** is a 2-dimensional robot consisting of 9 body parts and 8 joints connecting them (including two paws). The goal is to apply torque to the joints to make the cheetah run forward (right) as fast as possible, with a positive reward based on the distance moved forward and a negative reward for moving backward. The cheetah's torso and head are fixed, and torque can only be applied to the other 6 joints over the front and back thighs (which connect to the torso), the shins (which connect to the thighs), and the feet (which connect to the shins).

The **Ant** is a 3D quadruped robot consisting of a torso (free rotational body) with four legs attached to it, where each leg has two body parts. The goal is to coordinate the four legs to move in the forward (right) direction by applying torque to the eight hinges connecting the two body parts of each leg and the torso (nine body parts and eight hinges).

4. PPO Hyperparameter Tuning

We chose to tune clip coefficient (ε) and λ (in GAE) as these affect the algorithm's working to its core.

The clip coefficient controls the policy update size, this effectively dictates how quickly or slowly the policy is moving towards the optimal. Having the correct step size is important because if the step size is too small then the policy would take a lot of timesteps to reach the optimal or else if the step size is to big the policy would end up overshooting the optimal, both cases result in the formulation of a poor policy.

Tuning lambda controls the bias-variance trade off in estimating the advantage term. As lambda moves from 0 to 1 it cause the shift in advantage estimation being carried out as TD(0) at $\lambda = 0$ (high bias) and as Monte Carlo method at $\lambda = 1$ (high variance). Temporal Difference introduces bias as it bootstraps value estimates from the immediate next step. Monte Carlo methods are unbiased but inherently show high variance, this originates from variance being accumulated over the length of the episodes. Hence, requires the agent to be run on many episodes for the value estimates to reliably converge.

The following values of ε were chosen: $\varepsilon \in \{0.1, 0.2, 0.3\}$. We have explored values around the published default [1] ($\varepsilon = 0.2$) testing change in agent behaviour under a transition from a strict ($\varepsilon =$

0.1) to a permissive ($\varepsilon = 0.3$) policy update constraint. For λ the following values were chosen: $\lambda \in \{0.9, 0.95, 1.00\}$. Similar strategy of exploring around the published defualt [1] is followed, testing change in agent behaviour as advatage estimation moves from pure monte carlo at $\lambda = 1$ to sligthy towards TD(0) at $\lambda = 0.90$.

	$\boldsymbol{\lambda = 0.90}$	$\boldsymbol{\lambda = 0.95}$	$\boldsymbol{\lambda = 1.00}$
$\boldsymbol{\varepsilon = 0.1}$	1471.80	1378.69	1657.81
$\boldsymbol{\varepsilon = 0.2}$	**4449.39**	2706.83	2397.13
$\boldsymbol{\varepsilon = 0.3}$	1131.88	**4568.06**	150.90

Table 1 : Average episodic return in HalfCheetah-v4 environment with different γ and τ configurations averaged over 10 episodes using PPO algorithm

	$\boldsymbol{\lambda = 0.90}$	$\boldsymbol{\lambda = 0.95}$	$\boldsymbol{\lambda = 1.00}$
$\boldsymbol{\varepsilon = 0.1}$	2498.17	1419.31	11.57
$\boldsymbol{\varepsilon = 0.2}$	2277.72	3049.39	107.12
$\boldsymbol{\varepsilon = 0.3}$	**3204.60**	507.76	72.97

Table 2 : Average episodic return in Ant-v4 environment with different γ and τ configurations averaged over 10 episodes using PPO algorithm

5. SAC Hyperparameter Tuning

For the purpose of hyperparameter tuning, γ, the reward discount and tau, the polyak averaging constant were tuned.

γ plays a pivotal role in dictating the agents behaviour. It controls, to describe it intuitively, the patience of the agent. Having a larger γ forces to the agent to make more long sighted decisions that is give more weight to future rewards, on the other hand a smaller γ forces the agents to make short sighted decisions.

τ, or the polyak averaging constant, is used in calculating a moving average of the value network weights directly contribute to Q-function leanring. It weighs the contribution of current value network weights and target value network weights (previous moving average estimate). Essentially controlling the influence of immediate changes encountered as part of learning.

For hyperparameter tuning the number of training steps were reduced to 450k from 1000k(1 million).

The following values for γ were chosen: $\gamma \in \{0.90, 0.95, 0.99\}$. Here we have tested the change that arises from agressive discounting($\gamma = 0.90$) to long horizon planning ($\gamma = 0.99$). For τ we chose: $\tau \in \{0.05, 0.001, 0.005\}$. Here we have explored towards the upper range of τ relative to the published default [2] because exploring even lower values of τ on a limited training budget would be uninformative.

	$\boldsymbol{\gamma = 0.90}$	$\boldsymbol{\gamma = 0.95}$	$\boldsymbol{\gamma = 0.99}$
$\boldsymbol{\tau = 0.005}$	2294.81	2170.51	8556.36
$\boldsymbol{\tau = 0.01}$	2371.91	8707.92	8552.37
$\boldsymbol{\tau = 0.05}$	2385.95	2174.07	**9418.79**

Table 3 : Average episodic return in HalfCheetah-v4 environment with different γ and τ configurations averaged over 10 episodes using SAC algorithm

	$\gamma = 0.90$	$\gamma = 0.95$	$\gamma = 0.99$
$\tau = 0.005$	652.43	2143.56	
$\tau = 0.01$	892.43	1616.85	5016.65
$\tau = 0.05$	700.09	2290.87	3480.42

Table 4 : Average episodic return in Ant-v4 environment with different γ and τ configurations averaged over 10 episodes using SAC algorithm

6. Results and Comparison

Even though these methods work generally well and accomodate the above mentioned challenges, it is important to note that they individually work best for certain cases. PPO is feasible and effective in simulation-driven robotics workflows, where large amounts of data can be generated cheaply. In contrast, SAC is more feasible for real-world robotics applications. Its sample efficiency, robustness to noise, and ability to learn from off-policy data make it better suited for physical systems with limited interaction budgets, however it is tough to tune and is rather sensitive to hyperparameter tuning.

- SAC
  - **high γ**: locomotion tasks are long horizon planning tasks
  - **high τ**: performed better with incorporating immediate changes
- PPO
  - **high ε**: permissive policy update constraint
  - **mid λ**: exact monte carlo ($\lambda = 1$) is not good should just lean towards monte carlo

7. Proposed Robotics Task

- study POMDP

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *CoRR*, 2017, [Online]. Available: http://arxiv.org/abs/1707.06347

[2] T. Haarnoja *et al.*, "Soft Actor-Critic Algorithms and Applications," *CoRR*, 2018, [Online]. Available: http://arxiv.org/abs/1812.05905

[3] M. Towers *et al.*, "Gymnasium: A Standard Interface for Reinforcement Learning Environments." [Online]. Available: https://arxiv.org/abs/2407.17032